

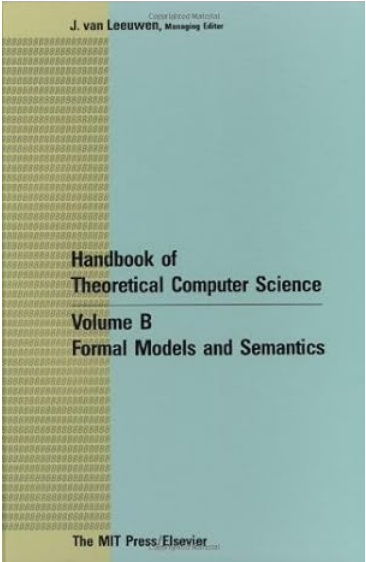
SI1001 Teoría de la Computación

Lambda cálculo

Andrés Sicard Ramírez

Universidad EAFIT

13 de marzo de 2025



CHAPTER 7

Functional Programming and Lambda Calculus

H. P. BARENDREGT
*Faculty of Mathematics and Computer Science, Catholic University Nijmegen,
Toernooiveld 1, 6525 ED Nijmegen, Netherlands*

Contents

1. The functional computation model	323
2. Lambda calculus	325
3. Semantics	337
4. Extending the language	347
5. The theory of combinators and implementation issues	355
Acknowledgment	360
References	360

Convenciones

- La numeración (de secciones, definiciones, teoremas, figuras, páginas, etc.) en estas diapositivas corresponde a la numeración en [Bar1992a].
- Los números naturales incluyen el cero, es decir, $\mathbb{N} = \{0, 1, 2, \dots\}$.
- El conjunto potencia de un conjunto A es denotado $\mathcal{P}A$.
- Los términos «definición inductiva» y «definición recursiva» se usan como sinónimos.
- En las definiciones inductivas se sobreentiende que el conjunto definido es el menor conjunto que satisface las condiciones.

Agradecimientos

Agradezco a mi colega Sergio Steven Ramírez Rico por señalarme algunas correcciones en una versión anterior de estas diapositivas.

Esquema de la presentación

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

- **Introducción**
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

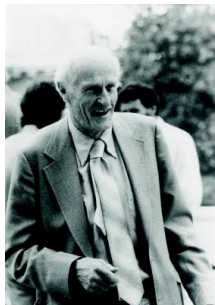
Alonzo Church (1903 – 1995)^a



^aFiguras tomadas de History of computers, Wikipedia y MacTutor History of Mathematics.

Introducción

Stephen Cole Kleene (1909 – 1994)^a



John Barkley Rosser (1907 – 1989)^b



^aFiguras tomadas de MacTutor History of Mathematics y Oberwolfach.

^bFigura tomada de la John Simon Guggenheim Memorial Foundation.

Introducción

- Un sistema formal creado por Alonzo Church alrededor de 1930 y sus estudiantes Stephen Kleene y J. Barkley Rosser.
- El objetivo inicial fue usar el lambda cálculo para las fundaciones de la matemáticas.
- Empleado para estudiar funciones y recursividad.
- Modelo de computabilidad.
- Base de los lenguajes de programación funcionales (Lisp, Haskell, ML, Scheme, etc.)
- Notación (*p. ej.* funciones anónimas y curryficación).

Bibliografía

- Artículos introductorios: [Bar1992b, § 2] y [BB2000].
- Una presentación sucinta de la teoría: [Sel2009, Apéndice A].
- Una presentación de los desarrollos actuales: [CH2009]
- Un artículo acerca de la historia del lambda cálculo: [Ros1984]. Véase también [CH2009] y [Sel2009].
- Libro de texto: [HS2008].
- La «biblia»: [Bar2004].

Introducción

Aplicación, λ -abstracción y β -conversión (informalmente)

En el tablero.

Introducción

Definición

Una función es de **orden superior** sii

- (i) recibe (al menos) una función como un argumento o
- (ii) retorna una función como resultado.

Introducción

Definición

Una función es de **orden superior** sii

- (i) recibe (al menos) una función como un argumento o
- (ii) retorna una función como resultado.

Ejemplos

En el tablero.

Introducción

Definición

Una función es de **orden superior** sii

- (i) recibe (al menos) una función como un argumento o
- (ii) retorna una función como resultado.

Ejemplos

En el tablero.

Observación

Las funciones de orden superior son usualmente llamadas «funcionales» en matemáticas.

Introducción

Funciones de varias variables

Funciones de varias variables pueden ser representadas por funciones de orden superior y usos repetidos de aplicación. Esta técnica fue propuesta por Schönfinkel, pero es llamada **curryficación** debido a que fue popularizada por Haskell Curry.

Introducción

Ejemplo (curryficación)

Sea $g : X \times Y \rightarrow Z$ una función de dos variables. Podemos definir dos funciones f_x y f por:

$$f_x : Y \rightarrow Z$$

$$f_x = \lambda y. g(x, y),$$

$$f : X \rightarrow (Y \rightarrow Z)$$

$$f = \lambda x. f_x.$$

Entonces $(f\ x)\ y = f_x\ y = g(x, y)$. Es decir, la función de dos variables

$$g : X \times Y \rightarrow Z$$

es representada por la función curryficada de orden superior.

$$f : X \rightarrow (Y \rightarrow Z).$$

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Lambda términos

Definición 2.1.1

Sea V un conjunto enumerable de variables. El conjunto de **lambda términos** o **λ -términos**, denotado por Λ , es inductivamente definido por:

$$x \in V \Rightarrow x \in \Lambda$$

(variable)

$$M, N \in \Lambda \Rightarrow (M N) \in \Lambda$$

(aplicación)

$$M \in \Lambda, x \in V \Rightarrow (\lambda x.M) \in \Lambda$$

(λ -abstracción)

Lambda términos

Ejemplos

En el tablero.

Lambda términos

Definición

El conjunto de λ -términos Λ también es definido frecuentemente empleando reglas de inferencia. Sea V un conjunto enumerable de variables, entonces:

$$\frac{x \in V}{x \in \Lambda} \text{ (variable)}$$

$$\frac{M \in \Lambda \quad N \in \Lambda}{(M \ N) \in \Lambda} \text{ (aplicación)}$$

$$\frac{M \in \Lambda \quad x \in V}{(\lambda x.M) \in \Lambda} \text{ (\lambda-abstracción)}$$

Lambda términos

Observación

Usualmente, el conjunto de λ -términos Λ es introducido por una gramática abstracta.^a

$\Lambda \ni t ::= x$	(variable)
$\quad \ t \ t$	(aplicación)
$\quad \ \lambda x. t$	(λ -abstracción)

^aVéase, p. ej. [Pie2002].

Lambda términos

Convenciones

- Las variables serán denotadas por letras minúsculas $x, y, z, \dots$
- Los λ -términos serán denotados por $L, M, N, \dots$

Notación

La expresión $M \equiv N$ denota que M y N son el mismo λ -término (identidad sintáctica).

Ligadura de variables

Definición

La ocurrencia de una variable x en un λ -término M es **libre** sii x **no está** en el alcance de λx .
En caso contrario, la ocurrencia de la variable x está **ligada** por la abstracción λx .

Definición 2.1.4.(i)

El **conjunto de variables libres** de un λ -término M , denotado por $FV(M)$, es inductivamente definido por:

$$FV : \Lambda \rightarrow \mathcal{P}V$$

$$FV(x) \quad := \{x\},$$

$$FV(M N) \quad := FV(M) \cup FV(N),$$

$$FV(\lambda x.M) \quad := FV(M) - \{x\}.$$

Sustitución

Definición

El resultado de **sustituir** cada ocurrencia libre de una variable x por un λ -término L en un λ -término M , cambiando variables libres para evitar su captura, denotado por $M[x \mapsto L]$, está definido por [HS2008, Definition 1.12]:

$$x[x \mapsto L] := L;$$

$$y[x \mapsto L] := y, \quad \text{si } y \neq x;$$

$$(M Q)[x \mapsto L] := (M[x \mapsto L]) (Q[x \mapsto L]);$$

$$(\lambda x.M)[x \mapsto L] := \lambda x.M;$$

$$(\lambda y.M)[x \mapsto L] := \lambda y.M, \quad \text{si } y \neq x \text{ y } x \notin \text{FV}(M);$$

$$(\lambda y.M)[x \mapsto L] := \lambda y.M[x \mapsto L], \quad \text{si } y \neq x, x \in \text{FV}(M) \text{ y } y \notin \text{FV}(L);$$

$$(\lambda y.M)[x \mapsto L] := \lambda z.(M[y \mapsto z])[x \mapsto L], \quad \text{si } y \neq x, x \in \text{FV}(M) \text{ y } y \in \text{FV}(L);$$

donde en la última ecuación, la variable z es seleccionada tal que $z \notin \text{FV}(L M)$.

Sustitución

Ejemplos

En el tablero.

Convenciones

- Los paréntesis más externos no son escritos.
- La aplicación tiene mayor precedencia que la λ -abstracción, es decir,

$$\lambda x. M N := (\lambda x. (M N)).$$

- La aplicación asocia a la izquierda, es decir,

$$M N_1 N_2 \dots N_k := (\dots ((M N_1) N_2) \dots N_k).$$

- La λ -abstracción asocia a la derecha, es decir,

$$\lambda x_1. \lambda x_2. \dots \lambda x_n. M := (\lambda x_1. (\lambda x_2. (\dots (\lambda x_n. M) \dots))).$$

Convenciones

Ejemplo

$$((((\lambda x. (\lambda y. (\lambda z. ((x z) (y z))))) u) v) w)$$

$\equiv$ (paréntesis externos son removidos)

$$(((\lambda x. (\lambda y. (\lambda z. ((x z) (y z))))) u) v) w$$

$\equiv$ (λ -abstracción asocia a la derecha)

$$(((\lambda x. \lambda y. (\lambda z. ((x z) (y z))))) u) v) w$$

$\equiv$ (λ -abstracción asocia a la derecha)

$$(((\lambda x. \lambda y. \lambda z. ((x z) (y z)))) u) v) w$$

$\equiv$ (aplicación mayor prec. que λ -abst.)

$$(((\lambda x. \lambda y. \lambda z. (x z) (y z)) u) v) w$$

$\equiv$ (aplicación asocia a la izquierda)

$$((\lambda x. \lambda y. \lambda z. (x z) (y z)) u) v w$$

$\equiv$ (aplicación asocia a la izquierda)

$$(\lambda x. \lambda y. \lambda z. (x z) (y z)) u v w$$

$\equiv$ (aplicación asocia a la izquierda)

$$(\lambda x. \lambda y. \lambda z. x z (y z)) u v w$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- **El lambda cálculo como una teoría formal**
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Introducción

- «*The principal object of study in the λ -calculus is the set of λ -terms modulo convertibility.*» [Bar2004, pág. 22].
- La relación de convertibilidad, denotada $=_{\text{conv}}$, es definida por la teoría λ .

La teoría λ

Definición

Los **términos** de la teoría λ son los λ -términos.

Definición

Las **fórmulas** de la teoría λ son de la forma $M =_{\text{conv}} N$, donde M, N son λ -términos.

Definición 2.1.5

- Axioma β

$$\frac{}{(\lambda x.M) N =_{\text{conv}} M[x \mapsto N]} (\beta)$$

- Axiomas y reglas de inferencia para la identidad

$$\frac{}{M =_{\text{conv}} M} (\text{refl}) \quad \frac{M =_{\text{conv}} N}{N =_{\text{conv}} M} (\text{sim}) \quad \frac{M =_{\text{conv}} N \quad N =_{\text{conv}} L}{M =_{\text{conv}} L} (\text{trans})$$

- Reglas de inferencia de compatibilidad

$$\frac{M =_{\text{conv}} N}{M L =_{\text{conv}} N L} (\text{appd}) \quad \frac{M =_{\text{conv}} N}{L M =_{\text{conv}} L N} (\text{appi}) \quad \frac{M =_{\text{conv}} N}{\lambda x.M =_{\text{conv}} \lambda x.N} (\text{abst})$$

Notación

Si $M =_{\text{conv}} N$ es un teorema es denotado por $\lambda \vdash M =_{\text{conv}} N$.

La teoría λ

Notación

Si $M =_{\text{conv}} N$ es un teorema es denotado por $\lambda \vdash M =_{\text{conv}} N$.

Definición 2.1.5

Los λ -términos M y N son **β -convertibles** sii $\lambda \vdash M =_{\text{conv}} N$.

La teoría λ

Notación

Si $M =_{\text{conv}} N$ es un teorema es denotado por $\lambda \vdash M =_{\text{conv}} N$.

Definición 2.1.5

Los λ -términos M y N son **β -convertibles** sii $\lambda \vdash M =_{\text{conv}} N$.

Observación

La teoría λ es

- una teoría ecuacional,
- *logic-free*, es decir, las fórmulas de la teoría no contienen constantes lógicas.

La teoría λ

Descripción

La demostración de un teorema en la teoría λ tiene forma de árbol, en donde las hojas son axiomas y la raíz es el teorema que se desea demostrar.

Ejemplo

Sean M y N dos λ -términos sin variables libres. Demostrar que $\lambda \vdash (\lambda x. \lambda y. x) M N =_{\text{conv}} M$.

(i) Una demostración en forma de árbol

$$\frac{\frac{\frac{}{(\lambda x. \lambda y. x) M =_{\text{conv}} \lambda y. M} (\beta)}{(\lambda x. \lambda y. x) M N =_{\text{conv}} (\lambda y. M) N} (\text{appd})}{(\lambda x. \lambda y. x) M N =_{\text{conv}} M} \frac{(\lambda y. M) N =_{\text{conv}} M (\beta)}{(\text{trans})}$$

Ejemplo

Sean M y N dos λ -términos sin variables libres. Demostrar que $\lambda \vdash (\lambda x. \lambda y. x) M N =_{\text{conv}} M$.

(i) Una demostración en forma de árbol

$$\frac{\frac{\frac{}{(\lambda x. \lambda y. x) M =_{\text{conv}} \lambda y. M} (\beta)}{(\lambda x. \lambda y. x) M N =_{\text{conv}} (\lambda y. M) N} (\text{appd})}{(\lambda x. \lambda y. x) M N =_{\text{conv}} M} \frac{(\lambda y. M) N =_{\text{conv}} M (\beta)}{(\text{trans})}$$

(ii) Una demostración lineal

1. $(\lambda x. \lambda y. x) M =_{\text{conv}} \lambda y. M$ (β)
2. $(\lambda x. \lambda y. x) M N =_{\text{conv}} (\lambda y. M) N$ $(\text{appd } 1)$
3. $(\lambda y. M) N =_{\text{conv}} M$ (β)
4. $(\lambda x. \lambda y. x) M N =_{\text{conv}} M$ $(\text{trans } 2, 3)$

Observación

Church empleó el nombre «conv» para denotar la relación de convertibilidad (véase, p. ej. [Chu1951]). El texto guía usa el símbolo de la igualdad «=», tal vez siguiendo a [CF1958, § 3.D.3].

Alfa congruencia

Definición

Un **cambio de variables ligadas** en un λ -término M es reemplazar un subtérmino $\lambda x.N$ de M por $\lambda y.(N[x \mapsto y])$ donde la variable y no ocurre en el λ -término N [Bar2004, Definición 2.1.11].

Definición

Un λ -término M es **α -congruente** con un λ -término N , denotado por $M \equiv_\alpha N$, si N resulta de M por un número finito (quizás cero) de cambios de variables ligadas [Bar2004, Definición 2.1.11].

Ejemplos

En el tablero.

Alfa congruencia

Teorema

La relación $\equiv_\alpha$ es una relación de equivalencia.^a

Convención

Nosotros sintácticamente identificamos λ -términos que son α -congruentes—siguiendo [Bar2004, Convención 2.1.12]—, es decir,

$$M \equiv N := M \equiv_\alpha N.$$

^aVéase, *p. ej.* [HS2008, Lema 1.19b].

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- **Combinadores**
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Definición 2.1.4.(ii)

Un **combinador** (o **λ -término cerrado**) es un λ -término sin variables libres.

Combinadores

Ejemplo 2.1.6 (algunos combinadores comunes)

$I := \lambda x. x$

(identidad)

$K := \lambda x. \lambda y. x$

(proyección primera componente)

$K_* := \lambda x. \lambda y. y$

(proyección segunda componente)

$S := \lambda f. \lambda g. \lambda x. f x (g x)$

(composición fuerte)

Combinadores

Ejemplo 2.1.6 (algunos combinadores comunes)

$I := \lambda x. x$

(identidad)

$K := \lambda x. \lambda y. x$

(proyección primera componente)

$K_* := \lambda x. \lambda y. y$

(proyección segunda componente)

$S := \lambda f. \lambda g. \lambda x. f x (g x)$

(composición fuerte)

Teorema

Sean L, M, N λ -términos, entonces

$$\lambda \vdash I M =_{\text{conv}} M,$$

$$\lambda \vdash K M N =_{\text{conv}} M,$$

$$\lambda \vdash K_* M N =_{\text{conv}} N,$$

$$\lambda \vdash S M N L =_{\text{conv}} M L (N L).$$

Combinadores

Observación

Los programas en un lenguaje de programación basado en el lambda cálculo son combinadores.
combinators.

Observación

Los combinadores **K** y **S** (es decir, la lógica combinatoria) son un lenguaje Turing-completo.

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- **Combinadores de punto fijo**
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Combinadores de punto fijo

Teorema 2.1.7.(i) (teorema de punto fijo)

Para cualquier λ -término F , existe un λ -término X , tal que

$$\lambda \vdash F X =_{\text{conv}} X.$$

Combinadores de punto fijo

Teorema 2.1.7.(i) (teorema de punto fijo)

Para cualquier λ -término F , existe un λ -término X , tal que

$$\lambda \vdash F X =_{\text{conv}} X.$$

Demostración

En el tablero.

Combinadores de punto fijo

Definición

Un **combinador de punto fijo** es un combinador **fix** tal que, para todos los λ -términos M ,

$$\lambda \vdash \text{fix } M =_{\text{conv}} M (\text{fix } M).$$

Combinadores de punto fijo

Teorema 2.1.7.(ii)

El combinador $Y := \lambda f. V V$, donde $V := \lambda x. f (x x)$, es un combinador de punto fijo.^a

^aDe acuerdo a [HS2008, pág. 36], este combinador fue insinuado por Curry en 1929 y publicado por primera vez por Rosenbloom [Ros1950]. Véase también [Bar2004, Corollary 6.1.3].

Combinadores de punto fijo

Teorema 2.1.7.(ii)

El combinador $Y := \lambda f. V V$, donde $V := \lambda x. f (x x)$, es un combinador de punto fijo.^a

Demostración

En el tablero.

^aDe acuerdo a [HS2008, pág. 36], este combinador fue insinuado por Curry en 1929 y publicado por primera vez por Rosenbloom [Ros1950]. Véase también [Bar2004, Corollary 6.1.3].

Combinadores de punto fijo

Teorema

El combinador $U\ U$, donde $U := \lambda u. \lambda x. x\ (u\ u\ x)$, es un combinador de punto fijo.^a

^aEste combinador fue definido por Turing [Tur1937]. Véase también [Bar2004, Definition 6.1.4].

Recursividad usando combinadores de punto fijo

Descripción

Los combinadores de punto fijo permiten representar la recursividad en el lambda cálculo.

Recursividad usando combinadores de punto fijo

Ejemplo

Ejemplo informal usando un combinador de punto fijo para definir la función factorial [Pey1987, § 2.4.1].

$$\begin{aligned}\text{fac} &:= \lambda n. \text{if } (n == 0) \text{ then } 1 \text{ else } n \times \text{fac } (n - 1) && \text{(combinador)} \\ &\equiv \lambda n. (\dots \text{fac } \dots) && \text{(combinador recursivo)} \\ &\equiv (\lambda f. \lambda n. (\dots f \dots)) \text{fac} && (\lambda\text{-abstracción en fac})\end{aligned}$$

Recursividad usando combinadores de punto fijo

Ejemplo

Ejemplo informal usando un combinador de punto fijo para definir la función factorial [Pey1987, § 2.4.1].

$$\begin{aligned}\text{fac} &:= \lambda n. \text{if } (n == 0) \text{ then } 1 \text{ else } n \times \text{fac } (n - 1) && \text{(combinador)} \\ &\equiv \lambda n. (\dots \text{fac } \dots) && \text{(combinador recursivo)} \\ &\equiv (\lambda f. \lambda n. (\dots f \dots)) \text{fac} && (\lambda\text{-abstracción en fac})\end{aligned}$$

Ahora, podemos redefinir la función factorial usando un combinador de punto fijo **fix**.

$$\text{h} := \lambda f. \lambda n. (\dots f \dots) \quad \text{(combinador no recursivo)}$$

$$\text{fac} := \text{fix h} \quad \text{(\text{fac} es un punto fijo de h)}$$

Recursividad usando combinadores de punto fijo

Ejemplo (continuación)

$$\begin{aligned}\text{fac } 1 &\equiv \text{fix } h \ 1 \\ &=_{\text{conv}} h \ (\text{fix } h) \ 1 \\ &\equiv (\lambda f. \lambda n. (\dots f \dots)) \ (\text{fix } h) \ 1 \\ &=_{\text{conv}} \text{if } (1 == 0) \text{ then } 1 \text{ else } 1 \times (\text{fix } h \ 0) \\ &=_{\text{conv}} 1 \times (\text{fix } h \ 0) \\ &=_{\text{conv}} 1 \times (h(\text{fix } h) \ 0) \\ &\equiv 1 \times ((\lambda f. \lambda n. (\dots f \dots)) \ (\text{fix } h) \ 0) \\ &=_{\text{conv}} 1 \times (\text{if } (0 == 0) \text{ then } 1 \text{ else } 1 * (\text{fix } h \ (-1))) \\ &=_{\text{conv}} 1 \times 1 \\ &=_{\text{conv}} 1\end{aligned}$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- **Codificación de Church**
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Definición 2.1.9.(i)

Para $n \in \mathbb{N}$ y $F, M \in \Delta$, definimos:

$$\begin{aligned} F^0(M) &:= M, \\ F^{n+1}(M) &:= F(F^n(M)). \end{aligned}$$

Codificación de Church

Definición

Los **numerales de Church**, denotados por c_n con $n \in \mathbb{N}$, codifican los números naturales:

$$c_n := \lambda f. \lambda x. f^n(x).$$

Codificación de Church

Ejemplo

Algunos numerales.

$$c_0 \equiv \lambda f. \lambda x. x,$$

$$c_1 \equiv \lambda f. \lambda x. f x,$$

$$c_2 \equiv \lambda f. \lambda x. f (f x),$$

$$c_3 \equiv \lambda f. \lambda x. f (f (f x)).$$

Codificación de Church

Sucesor

Un combinador para el sucesor es definido por

$$\text{succ} := \lambda n f x. f (n f x),$$

donde para todo $n \in \mathbb{N}$,

$$\lambda \vdash \text{succ } c_n =_{\text{conv}} c_{n+1}.$$

Codificación de Church

Proposición 2.1.10 (adición, multiplicación y exponenciación)

Sean

$$\text{add} := \lambda m. \lambda n. \lambda f. \lambda x. m f (n f x),$$

$$\text{mult} := \lambda m. \lambda n. \lambda f. \lambda x. m (n f) x,$$

$$\text{exp} := \lambda m. \lambda n. n m,$$

entonces para todo $m, n \in \mathbb{N}$,

$$\lambda \vdash \text{add } c_m c_n =_{\text{conv}} c_{m+n}.$$

$$\lambda \vdash \text{mult } c_m c_n =_{\text{conv}} c_{m \times n}.$$

$$\lambda \vdash \text{exp } c_m c_n =_{\text{conv}} c_{m^n}, \text{ excepto para } n = 0.$$

Codificación de Church

Definición 2.2.1

Codificamos los booleanos por

$$\text{true} := K, \quad \text{false} := K_*,$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash \text{true } M N =_{\text{conv}} M,$$

$$\lambda \vdash \text{false } M N =_{\text{conv}} N.$$

Codificación de Church

Test para cero

Un combinador para un test para cero es definido por

$$\text{isZero} := \lambda n. n (\lambda x. \text{false}) \text{true},$$

donde para todo $n \in \mathbb{N}$,

$$\lambda \vdash \text{isZero } c_0 =_{\text{conv}} \text{true},$$

$$\lambda \vdash \text{isZero } c_{n+1} =_{\text{conv}} \text{false}.$$

Codificación de Church

Predecesor

Un combinador para el predecesor es definido por

$$\text{pred} := \lambda n. \lambda f. \lambda x. n (\lambda g. \lambda h. h (g f)) (\lambda u. x) (\lambda u. u),$$

donde para todo $n \in \mathbb{N}$,

$$\lambda \vdash \text{pred } c_0 =_{\text{conv}} c_0,$$

$$\lambda \vdash \text{pred } c_{n+1} =_{\text{conv}} c_n.$$

Codificación de Church

Recursion

The factorial function is an example of recursive functions on natural numbers following the schemata

$$\begin{aligned}f &: \mathbb{N} \rightarrow A \\f\ 0 &= a \\f\ (S\ n) &= g\ n\ (f\ n)\end{aligned}$$

where A is a set, $a \in A$, $g : \mathbb{N} \rightarrow A \rightarrow A$ and S is the successor function.

Observación

A reader familiarised with recursion theory will identify the primitive recursion schemata.

Codificación de Church

A recursor for natural numbers

Let A be a set. From the previous schemata for (primitive) recursive functions, we define a recursor.^a

$$\text{rec} \quad : (\mathbb{N} \rightarrow A \rightarrow A) \rightarrow A \rightarrow \mathbb{N} \rightarrow A$$

$$\text{rec } f \ a \ 0 \quad = a$$

$$\text{rec } f \ a \ (S \ n) = f \ n \ (\text{rec } f \ a \ n)$$

^aA higher-order function which allow define recursive functions.

Codificación de Church

Recursor

Since `rec` is also a recursive function, we can define a recursor for numerals using `fix`.

$$\begin{aligned} h &:= \lambda r. \lambda f. \lambda a. \lambda n. (\text{isZero } n) a (f (\text{pred } n) (r f a (\text{pred } n))), \\ \text{rec} &:= \text{fix } h, \end{aligned}$$

where

$$\begin{aligned} \lambda \vdash \text{rec } f a c_0 &=_{\text{conv}} a, \\ \lambda \vdash \text{rec } f a c_{n+1} &=_{\text{conv}} f c_n (\text{rec } f a c_n). \end{aligned}$$

Codificación de Church

Recursor

Since `rec` is also a recursive function, we can define a recursor for numerals using `fix`.

$$\begin{aligned}h &:= \lambda r. \lambda f. \lambda a. \lambda n. (\text{isZero } n) a (f (\text{pred } n) (r f a (\text{pred } n))), \\ \text{rec} &:= \text{fix } h,\end{aligned}$$

where

$$\begin{aligned}\lambda \vdash \text{rec } f a c_0 &=_{\text{conv}} a, \\ \lambda \vdash \text{rec } f a c_{n+1} &=_{\text{conv}} f c_n (\text{rec } f a c_n).\end{aligned}$$

Pregunta

What is it necessary for defining `rec`? A fixed-point combinator, (implicit) «Boolean» case analysis, a test for zero and a predecessor function.

Codificación de Church

Ejemplo

A combinator for the factorial function

$$\text{fac} : \mathbb{N} \rightarrow \mathbb{N}$$

$$\text{fac } 0 = 1$$

$$\text{fac } (S\ n) = S\ n \times \text{fac } n$$

is defined by

$$\text{fac} := \text{rec } (\lambda x. \lambda y. \text{mult } (\text{succ } x) y) \text{ c}_1.$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- **Codificación de Barendregt**
- Representación de las funciones computables
- Referencias

Codificación de Barendregt

Definición 2.2.1

Codificamos los booleanos por

$$\text{true} := K, \quad \text{false} := K_*,$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash \text{true } M N =_{\text{conv}} M,$$

$$\lambda \vdash \text{false } M N =_{\text{conv}} N.$$

Codificación de Barendregt

Definición 2.2.2

Codificamos un par ordenado de M y N por:

$$[M, N] := \lambda z. z M N,$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash [M, N] \text{ true} =_{\text{conv}} M,$$

$$\lambda \vdash [M, N] \text{ false} =_{\text{conv}} N.$$

Codificación de Barendregt

Definición 2.2.3

Los **numerales de Barendregt**, denotados por $\overline{n}$, con $n \in \mathbb{N}$, codifican los números naturales:

$$\begin{aligned}\overline{0} &:= \mathbf{I}, \\ \overline{n+1} &:= [\mathbf{false}, \overline{n}].\end{aligned}$$

Codificación de Barendregt

Lema 2.2.4 (sucesor, predecesor y test para cero)

Sean

$$S^+ := \lambda x. [\text{false}, \bar{x}], \quad P^- := \lambda x. x \text{ false}, \quad \text{Zero} := \lambda x. x \text{ true},$$

entonces para todos $n \in \mathbb{N}$,

$$\lambda \vdash S^+ \bar{n} =_{\text{conv}} \overline{n+1},$$

$$\lambda \vdash P^- \bar{0} =_{\text{conv}} \bar{0},$$

$$\lambda \vdash P^- \overline{n+1} =_{\text{conv}} \bar{n},$$

$$\lambda \vdash \text{Zero} \bar{0} =_{\text{conv}} \text{true},$$

$$\lambda \vdash \text{Zero} \overline{n+1} =_{\text{conv}} \text{false}.$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- **Representación de las funciones computables**
- Referencias

Representación de las funciones computables

Definición

Una **función numérico-teórica** es una función

$$\mathbb{N}^p \rightarrow \mathbb{N}, \text{ con } p \in \mathbb{N}.$$

Representación de las funciones computables

Definición 2.2.5

Sea $f : \mathbb{N}^p \rightarrow \mathbb{N}$ una función numérico-teórica. La función f es **λ -definible respecto a la codificación de Barendregt** sii existe un combinador F , tal que para todo $n_1, \dots, n_k \in \mathbb{N}$,

$$\lambda \vdash F \overline{n_1} \dots \overline{n_p} =_{\text{conv}} \overline{f(n_1, \dots, n_p)}.$$

Representación de las funciones computables

Lema 2.2.9

Las funciones iniciales son λ -definibles (respecto a la codificación de Barendregt).

Lema 2.2.10

Las funciones λ -definibles (respecto a la codificación de Barendregt) son cerradas bajo composición.

Lema 2.2.11

Las funciones λ -definibles (respecto a la codificación de Barendregt) son cerradas bajo recursión primitiva.

Lema 2.2.12

Las funciones λ -definibles (respecto a la codificación de Barendregt) son cerradas bajo minimización.

Representación de las funciones computables

Teorema 2.2.13

Las funciones recursivas son λ -definibles (respecto a la codificación de Barendregt).

Representación de las funciones computables

Definición 2.2.5

Sea $f : \mathbb{N}^p \rightarrow \mathbb{N}$ una función numérico-teórica. La función f es **λ -definible respecto a la codificación de Church** sii existe un combinador F , tal que para todo $n_1, \dots, n_k \in \mathbb{N}$,

$$\lambda \vdash F c_{n_1} \dots c_{n_p} =_{\text{conv}} c_{f(n_1, \dots, n_p)}.$$

Representación de las funciones computables

Teorema 2.2.14

Las funciones recursivas son λ -definibles (respecto a la codificación de Church).

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Referencias

- [Bar2004] H. P. Barendregt. *The Lambda Calculus. Its Syntax and Semantics*. Revised edition, 6th impression. Vol. 103. Studies in Logic and the Foundations of Mathematics. Elsevier, 2004 (vid. págs. 10, 30, 40, 41, 51-53).
- [Bar1992a] Henk Barendregt. «Functional Programming and Lambda Calculus». En: *Handbook of Theoretical Computer Science. Volume B. Formal Models and Semantics*. Ed. por J. van Leeuwen. Second impression. MIT Press, 1992. Cap. 7. DOI: 10.1016/B978-0-444-88074-1.50012-3 (vid. pág. 3).
- [Bar1992b] Henk Barendregt. «Lambda Calculi with Types». En: *Handbook of Logic in Computer Science. Volume 2*. Ed. por S. Abramsky, Dov M. Gabbay y T. S. E. Maibaum. Clarendon Press, 1992, págs. 117-309 (vid. pág. 10).
- [BB2000] Henk Barendregt y Erik Barendsen. «Introduction to Lambda Calculus». Revisited edition. 2000 (vid. pág. 10).
- [CH2009] Felice Cardone y J. Roger Hindley. «Lambda-Calculus and Combinators in the 20th Century». En: *Handbook of Logic in Computer Science*. Ed. por Dov M. Gabbay y John Woods. Vol. 5. Elsevier, 2009, págs. 723-817 (vid. pág. 10).

Referencias

- [Chu1951] Alonzo Church. *The Calculi of Lambda-Conversion*. Second printing. Princenton University Press, 1951 (vid. pág. 39).
- [CF1958] Haskell B. Curry y Robert Feys. *Combinatory Logic*. Vol. I. North-Holland Publishing Company, 1958 (vid. pág. 39).
- [GW2009] Dov M. Gabbay y John Woods, eds. *Handbook of the History of Logic*. Vol. 5. Elsevier, 2009.
- [HS2008] J. Roger Hindley y Jonathan P. Seldin. *Lambda-Calculus and Combinators. An Introduction*. Cambridge University Press, 2008 (vid. págs. 10, 25, 41, 51, 52).
- [Pey1987] Simon L. Peyton Jones. *The Implementation of Functional Programming Languages*. Series in Computer Sciences. Prentice-Hall International, 1987 (vid. págs. 55, 56).
- [Pie2002] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002 (vid. pág. 21).
- [Ros1950] Paul C. Rosenbloom. *The Elements of Mathematical Logic*. Dover Publications, 1950 (vid. págs. 51, 52).

Referencias

- [Ros1984] J. Barkley Rosser. «Highlights of the History of Lambda-Calculus». En: *Annals of the History of Computing* 6.4 (1984), págs. 337-349. DOI: [10.1109/MAHC.1984.10040](https://doi.org/10.1109/MAHC.1984.10040) (vid. [pág. 10](#)).
- [Sel2009] Jonathan P. Seldin. «The Logic of Church and Curry». En: *Handbook of Logic in Computer Science*. Ed. por Dov M. Gabbay y John Woods. Vol. 5. Elsevier, 2009, págs. 819-873 (vid. [pág. 10](#)).
- [Tur1937] A. M. Turing. «The $\mathbf{p}$ -Function in λ - K -Conversion». En: *The Journal of Symbolic Logic* 4.2 (1937), [pág. 164](#). DOI: [10.2307/2268281](https://doi.org/10.2307/2268281) (vid. [pág. 53](#)).